

Rebuild Prompt

You are building a focused Google Apps Script project from an existing full production script.

Objective:

Create a reduced, focused production script that supports only the Dashboard/System Tools menu path and the functions needed for those menu items to compile and run.

Target version:

vGG_Current_Approved_Production_Script

vGG_Build_Validation_Report.txt

Required menu structure:

→ Master List Tools

→ Dashboard/System Tools

- ◆ Set up System Sheets
- ◆ Create / Refresh All Templates
- ◆ Quality
 - Dashboard Quality Start up
 - Dashboard Quality Validate Templates
 - Dashboard Quality Workflow
- ◆ Rebuild
 - Rebuild Format Dashboard Defaults

→ Sheet Management

- ◆ Templates
 - Hide Templates
 - Show Templates
- ◆ System Sheets
 - Show System Sheets
 - Hide System Sheets

Menu requirements:

- The script must create only the menu structure listed above.
- Do not include unused menu callbacks.
- Every menu item must point to an existing function.
- The menu labels must match exactly.

Required functional scope:

Include only the code required for the listed menu items and their direct dependencies.

System Sheet requirements:

The script must create or verify the Dashboard Quality Report.

The script must create or verify the Framework Timing Report.

System sheet setup must only create/verify sheet structure, section headers, anchors, formatting, frozen rows, widths, and protected layout as needed.

System sheet setup must be deterministic: every run produces the same sections, headers, formatting, anchors, widths, and colors.

Dashboard and Template Architecture Requirements:

The script must create and use a dashboard-driven template system.

Required dashboard sheet:

Format Dashboard

The Format Dashboard must contain configuration tables for:

1. Global Inputs

Used by Section A and template building.

Must include values such as:

- Header Row

- Data Start Row
- Freeze Rows
- Freeze Columns
- Default Date Format
- Standard Font
- Standard Font Size
- Template Version

2. Sheet Definitions

Used by Section B and Create / Refresh All Templates.

Must define each governed sheet:

- Sheet Type
- Report Title
- Template Name
- Base Color
- Hidden Template setting
- Uses Title Rows
- Uses Filter
- Uses Alternating Colors
- Uses Subheaders

3. Header Definitions

Used by Section C and template building.

Must define the headers for each governed sheet type.

Dashboard headers are the source of truth.

4. Column Definitions

Used by Section D and template formatting.

Must define each governed column:

- Column Header
- Width
- Alignment
- Wrap
- Number Format
- Visibility
- Date Column flag where applicable

5. Sheet Behavior Definitions

Used by Section E and template building.

Must define sheet-level behavior such as:

- Title rows
- Filters
- Frozen rows
- Frozen columns
- Alternating colors
- Subheaders
- Hidden template behavior

Template Requirements

Create / Refresh All Templates must build templates from Format Dashboard configuration.

Template names should come from Sheet Definitions.

Each template must:

- Use dashboard headers
- Apply dashboard column widths
- Apply dashboard alignment
- Apply dashboard wrap
- Apply dashboard number formats
- Apply dashboard hidden/visible column settings
- Apply dashboard sheet behavior settings
- Preserve title rows and section formatting where applicable

Templates are formatting sources for output sheets.

The script must not hardcode dashboard default values unless no dashboard exists and the user explicitly runs Rebuild Format Dashboard Defaults.

Rebuild Format Dashboard Defaults should create only the dashboard structure and headers needed for configuration entry. It should not require full historical default values.

Dashboard Quality Report requirements:

Below is the full Dashboard Quality Report test plan for sections **A–O**.

A — Global Inputs Verification

Purpose: confirm the dashboard has the required global settings needed to build templates.

Steps:

1. Load dashboard configuration.
2. Read required global input fields.

3. Check each required field is present.
4. Check each required field is not blank.
5. Check values are usable where required, such as row numbers, date format, font, or version.
6. Write only missing, blank, invalid, or unavailable settings.
7. If no issues exist, write one PASS summary row.

Headers:

None

Setting | Status | Issue | Quality Notes

B — Sheet Definitions Verification

Purpose: confirm every governed sheet used by the framework is defined on the dashboard.

Steps:

1. Load dashboard sheet definition table.
2. Build required sheet list from governed framework sheet types.
3. Check each required sheet type exists in dashboard definitions.
4. Check required fields exist, such as sheet type, report title, template name, and base color.
5. Report missing or incomplete sheet definitions.
6. If no issues exist, write one PASS summary row.

Headers:

None

Sheet | Status | Issue | Quality Notes

C — Sheet Headers Verification

Purpose: confirm dashboard header definitions are usable as the source of truth.

Steps:

1. Load dashboard header definition table.
2. Group headers by sheet type.

3. Check each governed sheet has at least one header.
4. Check for blank header names.
5. Check for duplicate header names within each sheet type.
6. Check required key headers exist where needed, such as PMR, name, date, and status fields.
7. Report only missing, blank, duplicate, or invalid headers.
8. If no issues exist, write one PASS summary row.

Headers:

None

Header | Status | Issue | Quality Notes

D — Column Definitions Verification

Purpose: confirm every governed column has the required build properties.

Steps:

1. Load dashboard column definition table.
2. For each dashboard header, find its column definition.
3. Check required properties: width, alignment, wrap, number format, visibility.
4. Validate width is numeric and greater than zero.
5. Validate alignment and wrap values are usable.
6. Validate date columns have date format where required.
7. Report only missing, blank, or invalid definitions.
8. If no issues exist, write one PASS summary row.

Headers:

None

Column | Status | Issue | Quality Notes

E — Sheet Behavior Verification

Purpose: confirm sheet-level behavior settings needed to build templates are present.

Steps:

1. Load dashboard behavior table.
2. Check each governed sheet has behavior settings.
3. Verify required behavior fields exist, such as title rows, filters, alternating colors, subheaders, hidden template, frozen rows, and frozen columns when governed.
4. Report only missing, blank, or invalid behavior fields.
5. If no issues exist, write one PASS summary row.

Headers:

None

Behavior | Status | Issue | Quality Notes

F — Template Structure & Validation

Purpose: confirm templates were created correctly from dashboard inputs.

Steps:

1. Load dashboard configuration.
2. For each governed sheet definition, locate its template.
3. Verify the template exists.
4. Verify template title rows exist where required.
5. Verify template header row exists.
6. Compare template headers to dashboard header definitions.
7. Identify missing template headers.
8. Identify unexpected template headers.
9. Validate required template sections or subsections.
10. For Monthly Change, verify subsection title row, spacer row, subsection header row, and formatting block.
11. Report template structure problems.
12. If no issues exist, write one PASS summary row.

Headers:

None

Template | Status | Issue | Quality Notes

G — Date Format Integrity Validation

Purpose: confirm Google Sheets preserved governed date formats.

Steps:

- 1. Load dashboard date-column definitions.
- 2. For each template, identify governed date columns.
- 3. Verify each date column exists on the template.
- 4. Read the actual number format from the template date column.
- 5. Confirm date format is present.
- 6. Sample template cells in the date column.
- 7. Confirm displayed values are not numeric serial numbers.
- 8. Confirm cells are not left as General format where a date format is required.
- 9. Report missing date columns, missing formats, general formats, serial-number display, or mismatches.
- 10. If no issues exist, write one PASS summary row.

Headers:

None

Column Header | Date Format Present | Status | Quality Notes

H — Dashboard Audit

Purpose: confirm the dashboard itself is structurally usable.

Steps:

- 1. Verify dashboard sheet exists.
- 2. Verify required dashboard sections or tables exist.
- 3. Verify expected dashboard anchors or headers exist.
- 4. Verify dashboard tables contain usable rows.
- 5. Verify dashboard configuration can be loaded without error.
- 6. Report missing sections, missing tables, unreadable ranges, or load errors.
- 7. If no issues exist, write one PASS summary row.

Headers:

None

Audit Item | Status | Issue | Quality Notes

I — Framework Health Check

Purpose: provide framework-wide health status.

Steps:

1. Verify the current script version is available.
2. Verify required menu callbacks exist.
3. Verify required system sheets exist.
4. Verify required templates exist or are marked not yet created.
5. Verify the timing system exists.
6. Verify Dashboard Quality sections A–O exist.
7. Verify no critical required helper is missing.
8. Report missing framework components or critical warnings.
9. If no issues exist, write one PASS summary row.

Headers:

None

Health Check | Status | Issue | Quality Notes

G — Signoff

Purpose: summarize readiness for release or operational use.

Steps:

1. Check whether Startup sections A–E have run.
2. Check whether Template Validation sections F–G have run.
3. Check whether Workflow sections H–O have run.
4. Check whether any section has FAIL or CRITICAL status.
5. Check whether timing contains unresolved bottlenecks.
6. Check whether required validation reports are present.
7. Write release-readiness status.
8. If not ready, write the required next action.

Headers:

None

Signoff Item | Status | Issue | Quality Notes

K — Summary

Purpose: roll up Dashboard Quality results.

Steps:

1. Read all Dashboard Quality section rows.
2. Count PASS rows.
3. Count WARNING rows.
4. Count FAIL rows.
5. Count CRITICAL rows.
6. Count NOT RUN rows.
7. Identify empty sections.
8. Identify the most recent update timestamp per section.
9. Write overall quality status.

Headers:

None

Metric | Value | Status | Quality Notes

L — Performance Summary

Purpose: summarize runtime performance from Framework Timing.

Steps:

1. Load Framework Timing Report.
2. Read the process summary.
3. Read a detailed timing log.
4. Identify slow, bottleneck, and critical steps.
5. Calculate total runtime by process.

6. Identify the worst severity by process.
7. Write open performance recommendations.
8. If the timing report is missing, write NOT AVAILABLE.

Headers:

None

Process | Runtime | Status | Quality Notes

M — Master List Health Check

Purpose: confirm Master List readiness and integrity.

Steps:

1. Locate the current Master List sheet.
2. Verify required Master List headers exist.
3. Verify Primary PMR Row column exists.
4. Verify required PMR/name/date/status fields exist.
5. Check for blank key fields where applicable.
6. Check for duplicate or missing PMR identifiers where applicable.
7. Confirm governed update tracking columns exist.
8. Report missing or invalid Master List conditions.
9. If full Master List processing is outside scope, write NOT RUN with explanation.

Headers:

None

Validation Item | Status | Issue | Quality Notes

N — Care Plan Sync Diagnostics

Purpose: confirm Care Plan synchronization readiness.

Steps:

1. Identify current Master List month.

2. Identify matching Care Plan Due sheet.
3. Identify matching Unlocked Care Plan sheet.
4. Verify required sync headers exist on each sheet.
5. Compare PMR or participant identifiers where available.
6. Identify missing Care Plan records.
7. Identify unmatched records.
8. Verify sync fields are available for writing or validation.
9. If Care Plan sheets are unavailable, write NOT AVAILABLE.
10. If full sync is outside scope, write NOT RUN with explanation.

Headers:

None

Sync Check | Status | Issue | Quality Notes

0 — Workflow & Synchronization Verification

Purpose: confirm major workflow and synchronization logic is functioning or available.

Steps:

1. Verify workflow callback functions exist.
2. Verify required synchronization helper functions exist.
3. Check contact processing availability.
4. Check phone split logic availability.
5. Check address combine logic availability.
6. Check language handling availability.
7. Check notes summary logic availability.
8. Check banner summary logic availability.
9. Check Care Plan sync availability.
10. Check Primary PMR assignment availability.
11. Report available, missing, not run, or not in scope.
12. If no workflow execution occurred, write NOT RUN with explanation.

Headers:

None

Workflow Check | Status | Issue | Quality Notes

Dashboard Quality execution ownership:

- Dashboard Quality Start up writes only Sections A through E.
- Dashboard Quality Validate Templates write only Sections F and G.
- Dashboard Quality Workflow writes only Sections H through O.
- No function should write to a section outside its ownership range.

Framework Tlming Report requirements:

Framework Timing Report Architecture

I would separate it into six functional sections.

Section A

Process Execution Summary

Purpose:

None

High-level timing summary of every framework process.

Headers:

Proces s	Runtime (Sec)	Statu s	Note s
-------------	------------------	------------	-----------

Examples:

Process	Runtime	Status	Notes
Set Up System Sheets	12.4	PASS	Completed
Dashboard Quality Start Up	48.3	WARNING	Slow
Create Templates	31.1	PASS	Completed

Functions Tested

None
Set Up System Sheets
Create / Refresh All Templates
Dashboard Quality Start Up
Dashboard Quality Validate Templates
Dashboard Quality Workflow
Rebuild Format Dashboard Defaults

Section B

Timing Severity Analysis

Purpose:

None

Convert runtimes into severity classifications.

Headers:

Processes	Runtime	Severity	Notes
-----------	---------	----------	-------

Suggested thresholds:

0-5 sec	PASS
5-15 sec	WARNING
15-30 sec	SLOW
30-60 sec	BOTTLENECK
60+ sec	CRITICAL

Example:

None

Dashboard Quality Start Up

429 sec

CRITICAL

Section C

Detailed Process Timing Log

Purpose:

None
Chronological execution log.

Headers:

| Timestamp | Process | Runtime | Status |

Example:

None
10:42:13

Show Templates

0.81

PASS

None

10:42:15

Create Templates

18.42

PASS

Required Logged Processes

Current framework should log:

None

Show Templates

Hide Templates

Show System Sheets

Hide System Sheets

Set Up System Sheets

Create / Refresh Templates

Dashboard Quality Start Up

Dashboard Quality Validate Templates

Dashboard Quality Workflow

Rebuild Dashboard Defaults

Section D

Performance Findings

Purpose:

None

Identify the slowest operations.

Headers:

| Finding | Status | Impact | Notes |

Examples:

None

Dashboard Quality Start Up

CRITICAL

429 seconds

None

Template Validation

BOTTLENECK

Exceeds target runtime

Logic

Sort processes by runtime.

Take:

None

Top 10 Slowest

Section E

Optimization Recommendations

Purpose:

None

Automatically identify optimization candidates.

Headers:

| Process | Recommendation | Priority | Notes |

Examples:

None

Dashboard Quality Start Up

Reduce repeated dashboard scans

HIGH

None

Template Validation

Cache header definitions

MEDIUM

Recommendation Rules

Repeated sheet scans:

None

Cache data

Repeated formatting:

None

Move to setup phase

Repeated header lookups:

None

Build lookup map

Section F

Timing Signoff

Purpose:

None

Determine if timing is acceptable.

Headers:

| Validation Item | Status | Notes | Action |

Checks:

Total Runtime

None

Below target?

Critical Findings

None

Any CRITICAL processes?

Bottlenecks

None

Any BOTTLENECK processes?

Timing Coverage

None

Did all required processes log timing?

Logging Integrity

None

Were timestamps captured?

Required Timing Functions

The Framework Timing Report should verify these functions specifically.

System Sheet Functions

None

Set Up System Sheets

Create Dashboard Quality Report

Create Framework Timing Report

Verify Dashboard Quality Structure

Verify Framework Timing Structure

Template Functions

None

Show Templates

Hide Templates

Create / Refresh All Templates

Validate Templates

Dashboard Quality Functions

None

Dashboard Quality Start Up

Dashboard Quality Validate Templates

Dashboard Quality Workflow

Dashboard Functions

None

Rebuild Format Dashboard Defaults

Functions NOT Timed

Do not include:

None

Individual helper functions

Cell formatting helpers

Color helpers

Column width helpers

Header writers

Utility functions

Only time:

None

Menu-accessible processes

Major framework operations

Long-running validation processes

What Framework Timing Should Reveal

When you open the report you should immediately know:

None

What ran?

What failed?

What is slow?

What is critical?

What should be optimized next?

Template requirements:

- Create / Refresh All Templates should build or refresh only dashboard-controlled templates needed by this focused script.
- Do not include dashboard default setting values unless required for compile/runtime.
- Do not include large hardcoded dashboard default tables unless strictly necessary.
- If the current script reads dashboard configuration from sheets, preserve that behavior.
- If defaults are only used to rebuild dashboard defaults, include only the minimum helper needed for the menu item to run.

Framework Timing requirements:

- The Framework Timing Report must capture runtimes for:

Set up System Sheets

Create / Refresh All Templates

Dashboard Quality Start up

Dashboard Quality Validate Templates

Dashboard Quality Workflow

Rebuild Format Dashboard Defaults

- The timing report must not rebuild itself during population.
- Timing data writers must not insert/delete rows.
- Timing data writers must write inside reserved blocks.

- Sheet Management menu items

Validation requirements:

Before delivering files, run:

1. Syntax scan
2. Brace balance check
3. Duplicate function declaration scan
4. Duplicate top-level const/let/class declaration scan
5. Menu callback audit
6. Missing dependency audit
7. Basic section registry audit for Dashboard Quality A-O
8. Timing callback audit

Validation report must include:

- PASS/FAIL syntax
- PASS/FAIL brace balance
- Number of functions
- Duplicate function list, if any
- Duplicate declaration list, if any
- Menu callbacks found/missing
- Missing dependencies, if any

- Dashboard Quality section list A-O
- Files generated

Execution requirement:

Do not stop after planning, audit, mapping, or dependency inventory.

Continue automatically through extraction, pruning, script build, validation, correction, and final file generation.

Stop only if:

- The uploaded/current script is unavailable
- A required source artifact is missing
- A governance conflict prevents continuation
- User approval is required to promote the script as the official production baseline

Final response:

Provide links to all generated files and a concise validation summary.